

ACTIVIDAD AUTÓNOMA N.º 09

Diferencias por plataforma y capacidades nativas

PharmaMobil sobre PharmaSoft

Estudiante	Juan Paredes Cárdenas
Asignatura	Desarrollo de Aplicaciones Móviles
Docente	Mg. Reyna Barreto Benjamin David
Unidad y sesión	Unidad 2, sesión 3 del curso (sesión 9)
Fecha	6 de octubre de 2026
Entrega indicada	12 de octubre de 2026, 23:59
Rama	feature/expect-actual-paredes

<https://github.com/DJuanParedes/pharmaMobil/tree/feature/expect-actual-paredes>

Inventario, informe comparativo, información del dispositivo y evidencias Android. El código compartido iOS pasó sus pruebas; las capturas iOS quedan pendientes porque el estudiante indicó no abrir su simulador.

Producto 1. Inventario de capacidades nativas

Rutas relativas a la raíz del repositorio. Se registran los cinco expect propios y, por separado, la interfaz Compartidor.

1. Plataforma original

Firma: `expect fun getPlatform(): Platform.`

commonMain: `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/Platform.kt.`

Android actual: `shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/Platform.android.kt.` API: `android.os.Build.VERSION.SDK_INT`; construye `AndroidPlatform`.

iOS actual: `shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/Platform.ios.kt.` API: `platform.UIKit.UIDevice.currentDevice.systemName()` y `systemVersion`; construye `IOSPlatform`.

2. Cliente HTTP

Firma: `expect fun crearHttpClient(): HttpClient.`

commonMain: `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/data/remote/HttpClientFactory.kt.`

Android actual: `shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/data/remote/HttpClientFactory.android.kt.` API/biblioteca: `motor Ktor CIO` sobre JVM; base `http://10.0.2.2:8080/api/v1/`.

iOS actual: `shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/data/remote/HttpClientFactory.ios.kt.` API/biblioteca: `motor Ktor Darwin` sobre `NSURLSession` de Foundation; base `http://localhost:8080/api/v1/`.

El pipeline, serialización, traducción de errores y repositorio REST son comunes.

3. Módulo de inyección

Firma: `expect val platformModule: Module.`

commonMain: `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/di/AppModule.kt.`

Android actual: `shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/di/PlatformModule.android.kt.` Koin `module, single<Compartidor>, androidContext()` y `CompartidorAndroid`.

iOS actual: `shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/di/PlatformModule.ios.kt.` Koin `module, single<Compartidor>` y `CompartidorIos`, sin Context Android.

Es infraestructura de inyección; Koin es una biblioteca, no una API del sistema.

Inventario de capacidades nativas (continuación)

4. Formato de moneda peruana

Firma: `expect fun formatearSoles(valor: Double): String.`

commonMain: `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.kt.`

Android actual: `shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.android.kt.` API JVM: `java.text.NumberFormat.getCurrencyInstance(java.util.Locale("es", "PE"))`.

iOS actual: `shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.ios.kt.` API Foundation: `NSNumberFormatterCurrencyStyle`, `NSNumberFormatter`, `NSLocale("es_PE")`, `NSNumber(double = valor)` y `stringFromNumber`.

Consumidores: `mapeo Producto.toUi()` y texto común para compartir.

5. Información del dispositivo — tercera capacidad

Firma: `expect class InfoDispositivo() { val sistema: String; val version: String }.`

commonMain: `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/platform/InfoDispositivo.kt.`

Android actual: `shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/InfoDispositivo.android.kt.` API: `android.os.Build.VERSION.RELEASE`; `sistema Android`.

iOS actual: `shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/platform/InfoDispositivo.ios.kt.` API: `platform.UIKit.UIDevice.currentDevice.systemName()` y `systemVersion`.

Interfaz conectada: `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/presentation/acerca/AcercaDeScreen.kt.` Destino `Acerca de en navigation/Screen.kt`; integrado por `App.kt`.

La capacidad original `getPlatform` utiliza `SDK_INT` en Android; la nueva utiliza la versión comercial `RELEASE` y campos separados, como pide la opción de la ficha.

6. Compartir producto — interfaz con inyección

Contrato: `interface Compartidor { fun compartir(texto: String) }.`

commonMain: `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/domain/platform/Compartidor.kt.`

Android implementación: `shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/CompartidorAndroid.kt.` API: `Intent.ACTION_SEND`, `Intent.EXTRA_TEXT`, `MIME text/plain`, `Intent.createChooser`, `FLAG_ACTIVITY_NEW_TASK`, `Context.startActivity`.

iOS implementación: `shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/platform/CompartidorIos.kt.` API: `UIActivityViewController`, escenas activas de `UIApplication`, `UIWindowScene`, `UIViewController` y extensión `UIKit popoverPresentationController`.

Texto común: `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/domain/usecase/TextoParaCompartir.kt.`

No existen actual para `Compartidor`: sus implementaciones cumplen la interfaz y se seleccionan mediante los actual de `platformModule`. Esta distinción evita atribuir al compilador la resolución de una dependencia que realiza Koin.

Producto 2. Informe comparativo

Extensión verificada: 734 palabras, incluidos títulos.

1. Formato monetario y resultados observados

En mi proyecto, `Formato.android.kt` delega en `NumberFormat.getCurrencyInstance(Locale("es", "PE"))`, mientras `Formato.ios.kt` utiliza `NSNumberFormatterCurrencyStyle`, `NSLocale("es_PE")` y `NSNumber`. Ambos identificadores representan español del Perú, pero seleccionan bibliotecas diferentes: Java en Android y Foundation en iOS. El locale expresa una preferencia cultural, no obliga a que sus motores compartan la misma versión de datos regionales ni la misma representación del espacio, símbolo y agrupación. Por eso elegí el mismo producto, Paracetamol 500 mg, con precio numérico 15.5 y stock 100, obtenido del backend oficial.

Android produjo literalmente «S/ 15.50». Su espacio entre símbolo y cifra es U+00A0, no separable. En iOS compilaron y pasaron las pruebas comunes, que normalizan ese espacio al verificar el texto. El resultado literal de su interfaz no está confirmado: detuve la ejecución visual cuando el estudiante indicó no abrir el simulador. Esta parte de la primera pregunta queda pendiente; no presentaré como observación un formato calculado en otro sistema. Incluso si ambas salidas coincidieran visualmente, sus implementaciones seguirían siendo diferentes. `Producto.toUi()` aplica el formato en presentación y conserva el `Double` del dominio. El composable únicamente muestra `ProductoUi.precio`.

2. Contexto Android y controlador iOS

`CompartidorAndroid` necesita un `Context` porque Android inicia una actividad mediante `Context.startActivity`. Koin entrega el contexto de aplicación registrado por `MainApplication`, no una `Activity` concreta. Por ello agregué `FLAG_ACTIVITY_NEW_TASK` al `Intent` creado por `Intent.createChooser`. El envío utiliza `ACTION_SEND`, `MIME text/plain` y `EXTRA_TEXT`; el selector permitió comprobar el nombre, precio y stock sin enviar el contenido a otra persona.

`CompartidorIos` no recibe ese objeto porque `UIKit` utiliza otra organización. Obtiene la escena activa de `UIApplication`, identifica la ventana principal y encuentra el controlador visible. Ese controlador presenta `UIActivityViewController`. Configuré además el ancla del popover para iPad e importé explícitamente la extensión correspondiente de `UIKit`. Koin construye aquí `CompartidorIos()` sin parámetros Android. Ninguna de estas decisiones aparece en `DetalleProductoViewModel`: el `ViewModel` sólo conoce la interfaz de dominio y el producto cargado.

3. Alternativa con interfaz e inyección

Si hubiera definido una interfaz `FormateadorMoneda`, habría registrado una implementación por plataforma en Koin e inyectado esa dependencia en los consumidores. Esto facilitaría sustituir el formato por un doble en pruebas y permitiría recibir configuración mediante el constructor. Sin embargo, aumentaría el número de contratos, registros y argumentos para una función sin estado. Además, olvidar un registro provocaría un fallo al resolver la dependencia, mientras una declaración actual ausente se detecta al compilar el target correspondiente.

Elegí `expect/actual` para `formatearSoles` y `InfoDispositivo`, porque sus implementaciones se seleccionan por `source set` y no requieren infraestructura adicional. Elegí interfaz con inyección para `Compartidor` porque Android necesita `Context` y las pruebas necesitan sustituir el selector. Las pruebas comunes usan un `Compartidor` que registra los textos, simula un fallo y verifica que el detalle conserve el producto al reintentar. El acoplamiento a `UIKit` y Android permanece aislado en sus `source sets`; la legibilidad del código común mejora al expresar la capacidad, sin reproducir sus mecanismos internos.

4. Ausencia deliberada de un actual

Comenté temporalmente el contenido de `Formato.android.kt` y ejecuté `:shared:compileAndroidMain`. El compilador informó exactamente: «Expected formatearSoles has no actual declaration in module <commonMain> for JVM». El log registra `:shared:compileAndroidMain FAILED` y `BUILD FAILED in 11s`. La captura muestra ese diagnóstico real mediante un visor del registro, no una pantalla de un IDE ni un mensaje fabricado. Después restauré los bytes originales y repetí la compilación, que finalizó correctamente. Esto demuestra una garantía por target: el actual iOS no puede satisfacer una compilación JVM. También deben coincidir el paquete, nombre, parámetros y retorno.

5. Capacidad que debe permanecer común

La validación de registrar un producto no debería trasladarse a `androidMain` ni `iosMain`. En `RegistrarProductoUseCase.kt`, `validarProducto` normaliza el nombre, comprueba su longitud de 3 a 150 caracteres, exige precio mínimo 0.01 y stock entero no negativo. Son reglas del inventario y del contrato REST, no funciones del sistema operativo. Duplicarlas permitiría que una plataforma aceptara productos rechazados por la otra y obligaría a corregir dos implementaciones.

Asimismo, `comoTextoParaCompartir()` conserva en `commonMain` la composición «nombre — precio · Stock: cantidad». Sólo el formato local y la apertura del selector dependen de la plataforma. Mi criterio consiste en separar reglas que describen el negocio de mecanismos que requieren una API nativa. La implementación de `Acerca de` confirma esa frontera: `Compose` muestra dos cadenas comunes y `InfoDispositivo` consulta `Build.VERSION.RELEASE` o `UIDevice`, según el sistema donde se ejecuta.

Producto 3. Información del dispositivo

Se eligió la opción Información del dispositivo, autorizada por la ficha. InfoDispositivo declara sistema y version como cadenas comunes. La implementación Android consulta Build.VERSION.RELEASE; iOS consulta UIDevice.currentDevice.systemName() y systemVersion.

AcercaDeScreen obtiene la información y la muestra en una pantalla accesible desde el menú Acerca de. Screen conserva los destinos anteriores y añade este destino. La interfaz no importa android.* ni platform.*.

La capacidad getPlatform preexistente no se presentó como nueva: el inventario distingue esa utilidad del nuevo expect class, que separa sistema y versión y muestra la versión comercial Android en lugar del nivel de API.

El APK Android final se construyó y ejecutó desde el código publicado. En iOS compilaron los fuentes compartidos y pasaron sus pruebas, pero la aplicación Swift y su flujo visual no se dan por verificados. Las capturas siguientes son originales de Android; la columna iOS informa el estado pendiente, sin reemplazar el dispositivo por una maqueta.

Interoperabilidad Kotlin-Swift

iOSApp.swift importa Shared y llama KoinlosKt.doinitKoinlos(); ContentView.swift integra MainViewControllerKt.MainViewController(). Los nombres se corresponden con los archivos Kotlin y la documentación de interoperabilidad; la inspección de una cabecera Shared.h generada para la aplicación queda pendiente. Los estados permanecen en Kotlin: Swift aloja la interfaz Compose y no reconstruye sus decisiones.

Las funciones suspend se exportan con completion handler y Swift puede importarlas como async. @Throws declara los errores que pueden cruzar. Las clases sealed pierden la exhaustividad de un enum Swift; las data classes se usan como clases exportadas, sin desestructuración Kotlin ni parámetros predeterminados de copy. Los anulables son opcionales y los primitivos pueden usar envoltorios Kotlin.

Procedencia y validación

Android: emulador API 37, sistema Android 17, capturas originales con ADB. iOS: pruebas Kotlin/Native ejecutadas previamente en macOS por la cuenta DJuanParedes. XCTest visual detenido por indicación del estudiante. Entrega individual, según la indicación del estudiante. Workflow: <https://github.com/DJuanParedes/pharmaMobil/actions/runs/37532482314>.

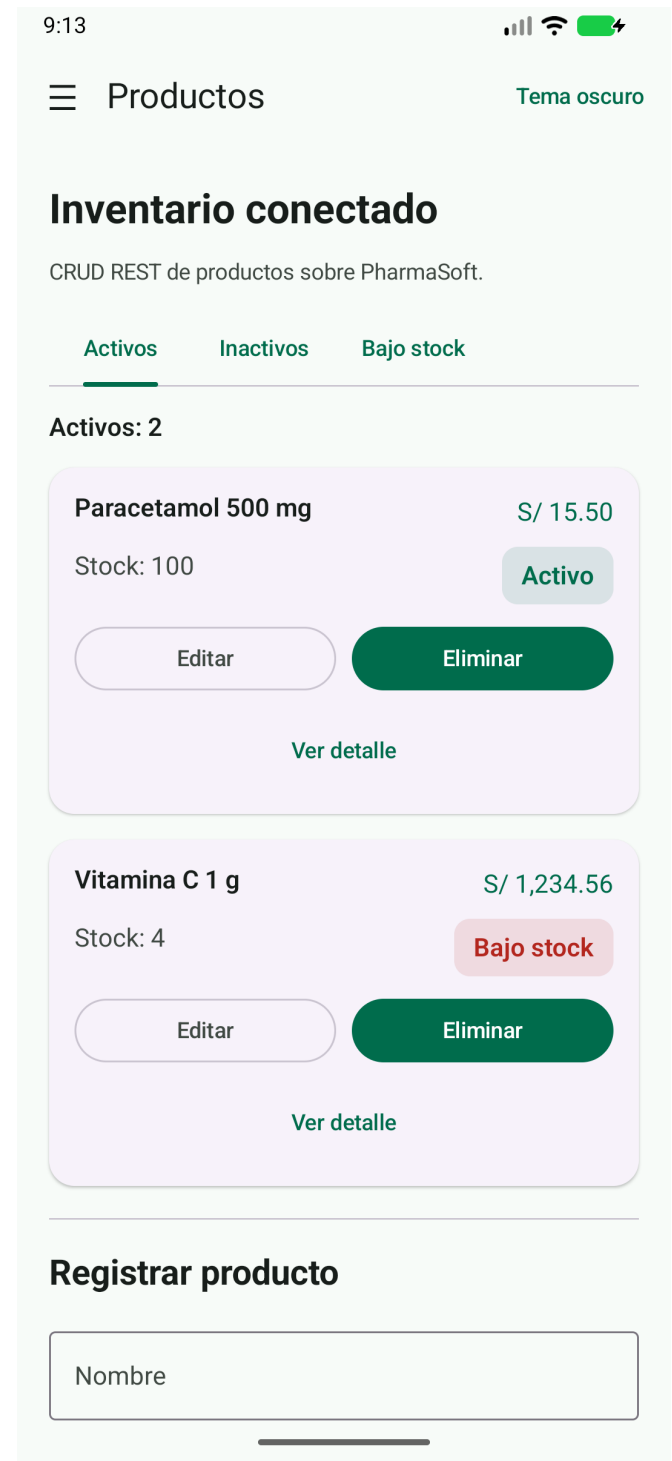
Backend oficial dreyna/pharmaSoft, commit 6bc94e321f993c194789e9ce9d008a48813e0bf2. POM de laboratorio con H2 modo Oracle, Flyway deshabilitado y Hibernate create-drop; controladores originales. Datos sintéticos insertados por POST: Paracetamol 500 mg (15.50, stock 100) y Vitamina C 1 g (1234.56, stock 4).

59 pruebas Android y 59 pruebas Kotlin iOS, cero fallos y errores. La prueba XCTest está preparada para comprobar listado, detalle, hoja nativa, copia del texto y Acerca de; ese recorrido no se completó. Los registros y resultados Kotlin iOS están conservados en evidencias/sesion9/ios. La búsqueda de imports nativos en todo commonMain obtuvo cero coincidencias.

Producto 4. Precio formateado, mismo producto

ANDROID

iOS

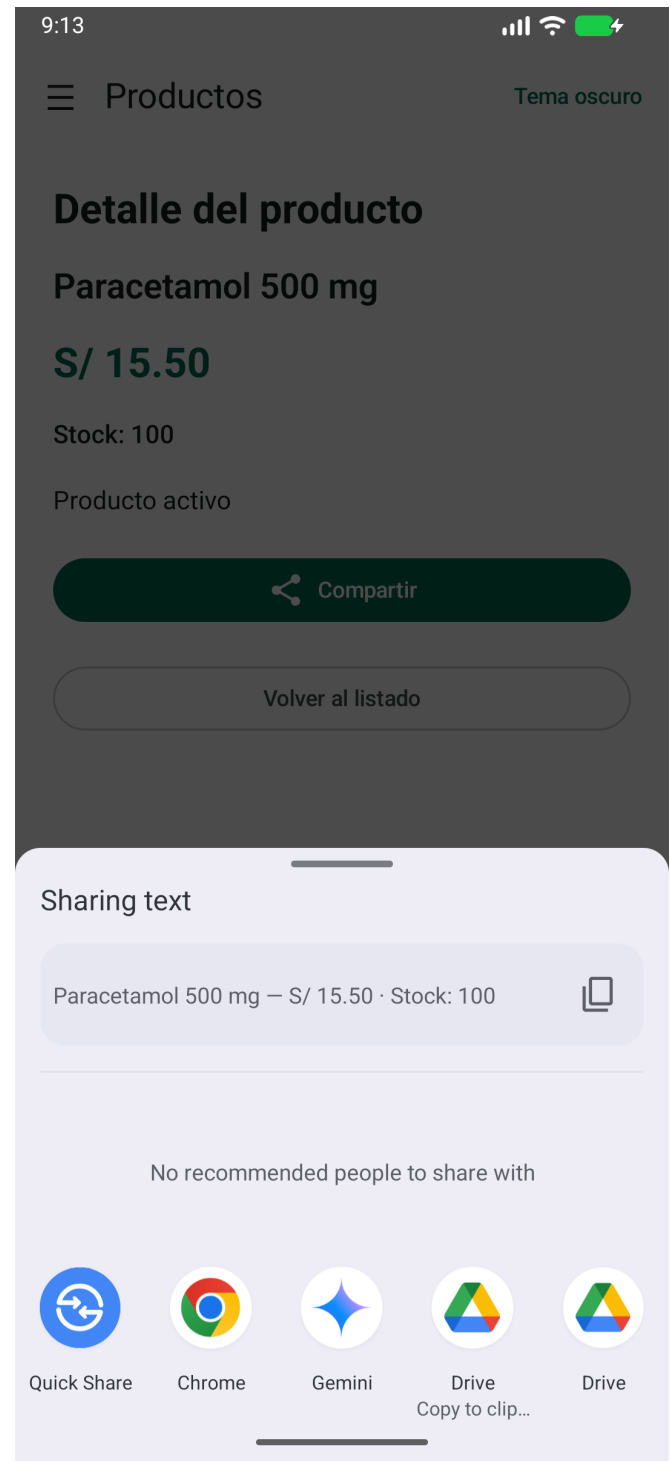


EVIDENCIA VISUAL iOS PENDIENTE El estudiante indicó no abrir el simulador. No se dispone de captura real de esta pantalla. Los actual iOS están implementados y las 59 pruebas Kotlin pasaron; esta observación visual queda pendiente.

Paracetamol 500 mg: precio 15.5 y stock 100 observados en Android. La comparación visual lado a lado con iOS queda pendiente. El resultado monetario se aplica al mapear a ProductoUi, fuera del dominio y del composible.

Producto 4. Selector y hoja de compartir

ANDROID



iOS

EVIDENCIA VISUAL iOS PENDIENTE El estudiante indicó no abrir el simulador. No se dispone de captura real de esta pantalla. Los actual iOS están implementados y las 59 pruebas Kotlin pasaron; esta observación visual queda pendiente.

Android presenta ACTION_SEND con el texto visible; no se envió el contenido a terceros. El actual iOS implementa UIActivityViewController, pero su hoja y el texto recibido no se observaron visualmente.

Producto 4. Tercera capacidad en la interfaz

ANDROID



iOS

EVIDENCIA VISUAL iOS PENDIENTE El estudiante indicó no abrir el simulador. No se dispone de captura real de esta pantalla. Los actual iOS están implementados y las 59 pruebas Kotlin pasaron; esta observación visual queda pendiente.

Acerca de muestra Android 17 en la ejecución capturada. El actual iOS consulta UIDevice; su resultado visible queda pendiente. La navegación y el contenido visible son Compose común.

Producto 4. Actual ausente: diagnóstico real

Se comentó temporalmente Formato.android.kt, se compiló y se restauró el archivo. La compilación posterior terminó correctamente. Captura de Chrome del emulador mostrando el log auténtico; no corresponde a un IDE.



Mensaje literal: Expected formatearSoles has no actual declaration in module <commonMain> for JVM

Repositorio, cotejo y referencias

Práctica: expect y control 1

<https://github.com/DJuanParedes/pharmaMobil/commit/322f9225a56d62685ff7ae65db0fd8107f2dfc3a>

Práctica: formatos nativos y ProductoUi

<https://github.com/DJuanParedes/pharmaMobil/commit/7563f2de34ce5015c5a0a3533f2a2a37875c9fd8>

Práctica: compartir, Koin y detalle

<https://github.com/DJuanParedes/pharmaMobil/commit/113d4242a745c69d718faf09abc047bbb695b21>

Producto 3: InfoDispositivo y Acerca de

<https://github.com/DJuanParedes/pharmaMobil/commit/8996ebbd5ac79c8d3772df900aa6378515308081>

Producto 1: inventario y README

<https://github.com/DJuanParedes/pharmaMobil/commit/abe6f13c411abd4a059c31748bf1642f9256a283>

Producto 2: informe comparativo de 734 palabras

<https://github.com/DJuanParedes/pharmaMobil/commit/e84685935664a0e4d48ebce4245127e5f3d89401>

Producto 4: capturas Android, error y validación JDK 21

<https://github.com/DJuanParedes/pharmaMobil/commit/e1b0956370a57ee546bb691f36b56aa97e77f2be>

Lista de cotejo de la actividad autónoma

Cumple: PDF con el nombre solicitado y portada completa

Cumple: Inventario con firmas, source sets, rutas y APIs

Cumple: Informe de 734 palabras, con cinco apartados y pendientes declarados

Cumple: Error literal provocado y actual restaurado

Cumple: Tercera capacidad: expect, dos actual y pantalla conectada

Cumple: Sin imports android.* ni platform.* en commonMain

Cumple: Capturas Android de las tres capacidades y del error

Cumple: README con Código específico de plataforma

Cumple: Commits propios publicados en la rama de entrega

Pendiente: capturas de las tres capacidades en iOS, resultado monetario literal iOS de la pregunta 1, comprobación visual de Koin y compartir, y compilación de la aplicación Swift. El estudiante indicó no abrir el simulador; no se declara cumplida la evidencia en ambas plataformas.

Entrega individual de Juan Paredes Cárdenas. El PDF y el enlace de su rama están disponibles; antes de presentar la rúbrica como completamente satisfecha deben resolverse los puntos iOS pendientes.

Referencias

Guía práctica N.º 09 y Actividad Autónoma N.º 09: Documentos del curso proporcionado/localizado, 6 de octubre de 2026.

Kotlin: expected and actual declarations: <https://kotlinlang.org/docs/multiplatform/multiplatform-expect-actual.html>

Kotlin: interoperability with Swift/Objective-C: <https://kotlinlang.org/docs/native-objc-interop.html>

Código y registros propios: <https://github.com/DJuanParedes/pharmaMobil/tree/feature/expect-actual-paredes>